



InfiniFi Protocol

Security Review



Disclaimer

Security Review

InfiniFi Protocol



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

InfiniFi Protocol



Table of Contents

Disclaimer	3
Summary	7
Scope	9
Methodology	9
Project Dashboard	13
Risk Section	16
Findings	18
3S-Protocol-M01	18
3S-Protocol-M02	20
3S-Protocol-L01	22
3S-Protocol-L02	24
3S-Protocol-N01	26
3S-Protocol-N02	27
3S-Protocol-N03	28
3S-Protocol-N04	30
3S-Protocol-N05	31

Summary Security Review

InfiniFi Protocol



Summary

Three Sigma audited InfiniFi in a 0.8 person week engagement. The audit was conducted from 19/01/2026 to 20/01/2026.

Protocol Description

InfiniFi is a DeFi protocol that enables users to mint and redeem receipt tokens (iUSD, iETH) against collateral assets (USDC, ETH). The protocol features a sophisticated yield generation system through multiple farm integrations, a locking mechanism for enhanced rewards, and a governance system for farm allocation voting.

Scope Security Review

InfiniFi Protocol

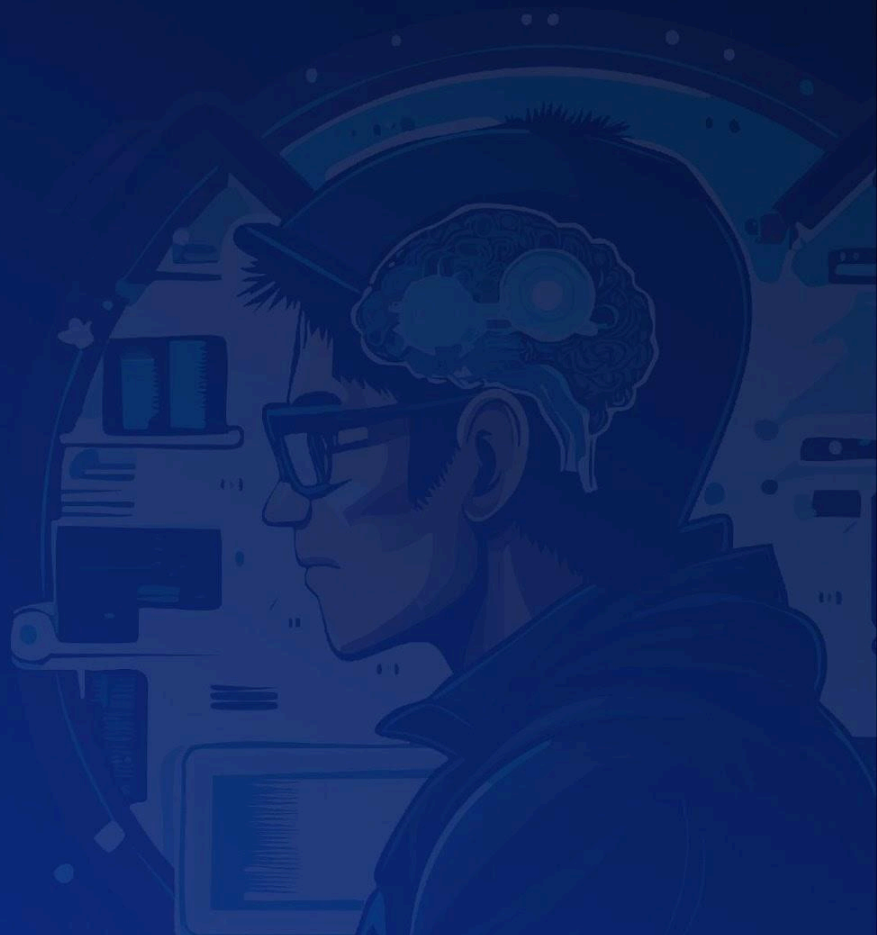


Scope

Filepath	nSLOC
R3:src/integrations/farms/GauntletAlphaFarm.sol	151
R2:src/integrations/farms/MidasFarm.sol	115
R1:src/integrations/farms/MapleFarm.sol	106
R4:src/integrations/RWAEscrow.sol	55
R4:src/integrations/farms/RWAEscrowFarm.sol	36
Total	463

Methodology Security Review

InfiniFi Protocol



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on ImmuneFi's [Vulnerability Severity Classification System \(v2.3\)](#) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard Security Review

InfiniFi Protocol



Project Dashboard

Application Summary

Name	InfiniFi
Repository	R1: https://github.com/InfiniFi-Labs/infinifi-contracts/pull/283 R2: https://github.com/InfiniFi-Labs/infinifi-contracts/pull/284 R3: https://github.com/InfiniFi-Labs/infinifi-contracts/pull/287 R4: https://github.com/InfiniFi-Labs/infinifi-contracts/pull/288
Commit	R1 - ac1919c R2 - 0a3b9f1 R3 - 0e0d736 R4 - 7eca9e5
Language	Solidity
Platform	EVM

Engagement Summary

Timeline	19/01/2026 to 20/01/2026
Nº of Auditors	2
Review Time	0.8 person weeks

Vulnerability Summary

Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	2	2	0
Low	2	2	0
None	5	4	1

Category Breakdown

Suggestion	5
Documentation	0
Bug	4
Optimization	0
Good Code Practices	0

Risk Section Security Review

InfiniFi Protocol



Risk Section

- Temporary Asset Accounting Discrepancy:** In the `MidasFarm::assets` function, pending request values (both deposits and redemptions) are included in `totalAssets` calculations. When a Midas admin rejects a request, the status changes to `Canceled`, and the admin must manually transfer the asset/`mToken` back to the contract. During the interim period between rejection and manual transfer completion, the `assets` function will report an understated value. While on-chain transaction analysis shows no instances of admin rejections to date, this edge case should be considered when relying on the `assets` function for accurate accounting, as there is no mechanism to verify whether the returned asset/`mToken` has arrived at the contract.
- Asset Conversion Decimal Consistency:** The `MidasFarm` contract relies on the `MultiAssetFarmV2::convert` function for asset conversion calculations. To ensure correctness of these conversions, the `convert` function must return output values denominated in `_tokenOut`'s decimals. Failure to maintain this decimal consistency could result in incorrect asset valuations, potentially leading to accounting discrepancies or unintended financial outcomes within the system.
- Temporary Denial of Service via Token Configuration:** Gauntlet administrators have the capability to disable asynchronous deposits and redemptions for any token through `Provisioner.setTokenDetails` by modifying `TokenDetails.asyncDepositEnabled` and `TokenDetails.asyncRedeemEnabled` flags. Additionally, the `Provisioner.removeToken` function can produce similar effects. These actions could result in a temporary lock of user funds. However, this risk is mitigated by the `refundRequest` mechanism, which allows the farm to reclaim funds after the request deadline has passed.

Findings

Security Review

InfiniFi Protocol



Findings

3S-Protocol-M01

Shared **maxSlippage** for deposit and redeem incompatible with distinct Provisioner fees

Id	3S-Protocol-M01
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #6f0568d .

Description

The **GauntletAlphaFarm** contract integrates with Gauntlet's **Provisioner**, which applies fee multipliers to deposit and redeem operations via the **TokenDetails** struct. These multipliers are defined separately as **depositMultiplier** and **redeemMultiplier**, expressed in basis points where 9900 represents a 1% fee.

The **GauntletAlphaFarm** constructor initializes a single **maxSlippage** variable to **0.999e18** (99.9%), which is then applied identically in both **GauntletAlphaFarm::vaultRequestDeposit** and **GauntletAlphaFarm::vaultRequestRedeem** when calculating **minUnitsOut** and **minTokensOut** respectively.

This design presents two issues. The Provisioner allows **depositMultiplier** and **redeemMultiplier** to be configured independently, yet the farm enforces the same slippage tolerance for both operations. If one fee differs from the other, the single parameter cannot accommodate both correctly.

More immediately, with USDC multipliers currently set at 1% for both operations, the Provisioner returns only 99% of the calculated output. Since **maxSlippage** demands at least 99.9%, the minimum output requirement exceeds what the Provisioner can provide. Consequently, all deposit and redeem requests will fail when solved because the actual output falls below the specified minimum threshold. Gauntlet's own documentation suggest using a 3% slippage tolerance.

Recommendation

Introduce separate slippage parameters for deposit and redeem operations, such as **maxDepositSlippage** and **maxRedeemSlippage**, to accommodate the Provisioner's asymmetric fee structure. Set default values in accordance with Gauntlet's recommended 3% slippage tolerance (**0.97e18**) to account for both the fee multipliers and potential price fluctuations.

3S-Protocol-M02

Single slippage parameter for asymmetric vault fees leads to suboptimal configuration or failed transactions

Id	3S-Protocol-M02
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #18bffd9 .

Description

The **MidasFarm** contract integrates with Midas protocol's deposit and redemption infrastructure through two distinct vault contracts: **depositVault** for minting mTokens and **redeemVault** for burning them. Each Midas vault inherits from **ManageableVault** which maintains independent fee configurations through the **instantFee** state variable and per-token fees stored in **tokenFee** mappings. The **_getFeeAmount** function in **ManageableVault** combines these values when **isInstant** is true, allowing each vault to charge different percentages for instant operations.

In **MidasFarm**, both **vaultDepositInstant** and **vaultRedeemInstant** rely on the same **maxSlippage** storage variable to compute **minReceiveAmount**. Since Midas configures instant deposit fees and instant redemption fees independently, a single slippage tolerance cannot accommodate both operations. This asymmetry is demonstrated in **IntegrationTestMidasFarm::testRedeemInstant**, where instant redemptions incur a 0.5% fee while instant deposits in **testDepositInstant** do not. The test shows that with the default **maxSlippage** of **0.99999e18**, calling **vaultRedeemInstant** reverts with **"RVS: minReceiveAmount > actual"**, and only succeeds after manually adjusting **maxSlippage** to **0.994e18**. Setting **maxSlippage** tight enough for deposits causes redemptions to fail, whereas loosening it for redemptions leaves deposits unprotected against unexpected slippage.

Recommendation

Replace the single **maxSlippage** variable with two dedicated parameters, one for each instant operation.

- uint256 public maxSlippage;
- + uint256 public maxSlippageDeposit;
- + uint256 public maxSlippageRedeem;

Update **vaultDepositInstant** to use **maxSlippageDeposit** and **vaultRedeemInstant** to use **maxSlippageRedeem**. Provide corresponding setter functions restricted to **CoreRoles.PROTOCOL_PARAMETERS** for each variable.

3S-Protocol-L01

Incorrect address parameter in **TokensWithdrawn** event emission leads to inaccurate off-chain data

Id	3S-Protocol-L01
Classification	Low
Impact	Low
Likelihood	High
Category	Bug
Status	Addressed in #a053037 .

Description

The **RWAEscrow** contract serves as an intermediary that holds and manages asset tokens between a farm contract and an external RWA receiver address. The **RWAEscrow::withdraw** function allows the owner to pull tokens back from the escrow to the farm when sufficient liquidity is available.

Within **withdraw**, the function correctly transfers tokens to **msg.sender** via **safeTransfer(msg.sender, _amount)**. However, the subsequent **TokensWithdrawn** event emission incorrectly logs **receiver** as the recipient address instead of **msg.sender**. The **receiver** variable is an immutable address representing the external RWA destination, which is unrelated to the withdrawal operation. This discrepancy causes the emitted event to report a recipient that did not actually receive the tokens. Off-chain indexers, monitoring systems, and accounting tools that rely on event data will record incorrect withdrawal information, potentially leading to reconciliation failures and difficulties in debugging token flows.

Recommendation

Update the event emission to use **msg.sender** to accurately reflect the actual token recipient.

```
function withdraw(uint256 _amount) external onlyOwner {
    totalAssets -= _amount;
    lastUpdatedAt = block.timestamp;
    ERC20(assetToken).safeTransfer(msg.sender, _amount);
    - emit TokensWithdrawn(block.timestamp, receiver, _amount);
```

```
+ emit TokensWithdrawn(block.timestamp, msg.sender, _amount);  
}
```

3S-Protocol-L02

Pending deposit request assets calculation uses pre-fee amount instead of post-fee amount

Id	3S-Protocol-L02
Classification	Low
Impact	Low
Likelihood	High
Category	Bug
Status	Addressed in #5ec2909 .

Description

The **MidasFarm** contract integrates with Midas vaults to handle deposit and redemption operations. The **assets()** function calculates the total value held by the farm, including any pending deposit or redemption requests that have not yet been processed. This function is used for accounting purposes to accurately represent the farm's holdings.

When a deposit request is created in the Midas **DepositVault**, the vault stores two USD amounts in the request struct:

- **depositedUsdAmount**: The total USD value of the deposited tokens (before fees are deducted)
- **usdAmountWithoutFees**: The USD value that will actually be converted to mTokens (after fees are deducted)

The **MidasFarm::assets()** function currently uses **depositRequest.depositedUsdAmount** to calculate the value of pending deposit requests. However, this represents the gross deposit amount before the Midas vault deducts its fees. The actual value that the farm will receive in mTokens is based on **usdAmountWithoutFees**, which is the net amount after fees.

This causes the **assets()** function to overstate the farm's total assets by the fee amount whenever there is a pending deposit request.

Note: This issue assumes the fix from the previous report (separating **DepositRequest** and **RedeemRequest** struct types) has been applied. With that fix in place, the redemption request correctly uses **amountMToken**, which already represents the post-fee amount.

Recommendation

Use **usdAmountWithoutFees** instead of **depositedUsdAmount** when calculating the value of pending deposit requests.

```
// MidasFarm.sol - assets() function
    // add pending deposit request amount if still pending
    uint256 _pendingDepositRequestId = pendingDepositRequestId;
    if (_pendingDepositRequestId != 0) {
        IMidasManageableVault.DepositRequest memory depositRequest =

IMidasDepositVault(depositVault).mintRequests(_pendingDepositRequestId);
        if (depositRequest.status ==
IMidasManageableVault.RequestStatus.Pending) {
-            // deposit amount is in assetToken but normalized to 18 decimals
-            totalAssets += depositRequest.depositedUsdAmount /
decimalsScalingFactor;
+            // use amount after fees since that's what will be converted to mTokens
+            totalAssets += depositRequest.usdAmountWithoutFees /
decimalsScalingFactor;
        }
    }
```

3S-Protocol-N01

Missing zero-value validation in **setMaxPriceAge** and **setDeadline**

Id	3S-Protocol-N01
Classification	None
Category	Suggestion
Status	Acknowledged

Description

The **GauntletAlphaFarm** contract allows privileged addresses with the **PROTOCOL_PARAMETERS** role to configure two timing parameters via **GauntletAlphaFarm::setDeadline** and **GauntletAlphaFarm::setMaxPriceAge**. The **deadline** parameter determines how long a request remains valid before expiring, while **maxPriceAge** defines the maximum acceptable age for price data when processing requests.

Neither function validates that the provided value is non-zero. When **vaultRequestDeposit** or **vaultRequestRedeem** is called, these parameters are passed directly to the Gauntlet **PROVISIONER**:

```
PROVISIONER.requestDeposit(
    IERC20(assetToken), _assetsIn, minUnitsOut, 0, block.timestamp + deadline,
    maxPriceAge, false
);
```

If **deadline** is set to zero, the request deadline becomes **block.timestamp**, causing immediate expiration. If **maxPriceAge** is set to zero, any price data is considered stale since no price update can occur within a zero-second window. Either misconfiguration renders all deposit and redeem requests unprocessable until the parameter is corrected, effectively freezing the farm's ability to interact with the Gauntlet vault.

Recommendation

Add zero-value checks to both setter functions.

3S-Protocol-N02

Misleading NatSpec comment in **assets()** leads to incorrect understanding of managed asset accounting

Id	3S-Protocol-N02
Classification	None
Category	Suggestion
Status	Addressed in #d6c6c47 .

Description

The **RWAEscrowFarm** contract is designed such that any USDC held directly by the farm is considered a donation rather than part of the managed assets. This design intent is documented in the contract header at lines 12-13, which states that any USDC held in the farm "will be considered as a welcome donation."

The **assets()** function at lines 30-32 correctly implements this design by returning only **escrow.totalAssets()**, excluding any tokens held directly by the farm contract. However, the NatSpec comment at line 29 states:

/// @notice Returns the asset token balance of this farm + escrow total assets

This comment is inaccurate, and this documentation mismatch can mislead developers and integrators into believing that **RWAEscrowFarm::assets()** accounts for tokens held directly by the farm, potentially causing incorrect assumptions about the protocol's asset accounting logic.

Recommendation

Update the NatSpec comment to accurately reflect the function's behavior:

- **/// @notice Returns the asset token balance of this farm + escrow total assets**
 + **/// @notice Returns the escrow total assets (tokens held directly by this farm are excluded)**

3S-Protocol-N03

Redundant **checkSlippage** modifier in **GauntletAlphaFarm** request functions

Id	3S-Protocol-N03
Classification	None
Category	Suggestion
Status	Addressed in #6f0568d .

Description

The **GauntletAlphaFarm** contract integrates with the Gauntlet Provisioner to handle asynchronous deposits and redemptions. The contract tracks pending requests through a hash-based system, where submitted requests are stored in the **hashes** enumerable set and their details are recorded in the **requests** mapping.

The **checkSlippage** modifier, inherited from **Farm**, is designed to protect against value loss during operations by comparing the farm's total assets before and after execution.

If assets decrease beyond the **maxSlippage** threshold, the transaction reverts. This modifier is applied to **vaultRequestDeposit**, **vaultRequestRedeem**, and **vaultRequestRefund**. However, it provides no protection in these functions because the **assets()** calculation inherently accounts for pending requests, making **assetsBefore** always equal to **assetsAfter**.

The **assets()** function calculates total value by summing:

1. The **assetToken** balance held directly by the contract
2. The vault shares held directly by the contract (converted to asset value)
3. Tokens locked in pending deposit requests via **requests[hash].tokens**
4. Units locked in pending redeem requests via **requests[hash].units** (converted to asset value)

The **checkSlippage** modifier is redundant for each function because:

- **`vaultRequestDeposit`**: Tokens sent to **PROVISIONER** are offset by storing the same amount in **requests[hash].tokens**, which **assets()** includes in its calculation.
- **`vaultRequestRedeem`**: Vault shares sent to **PROVISIONER** are offset by storing the same amount in **requests[hash].units**, which **assets()** includes in its calculation.

- ``vaultRequestRefund``: Tokens or shares returned from **PROVISIONER** are offset by `_pruneHashes()` removing the corresponding `requests[hash]` entry.

In all cases, `assetsAfter == assetsBefore`, so the slippage check always passes trivially.

Recommendation

Remove the `checkSlippage` modifier from the three request functions:

```
function vaultRequestDeposit(uint256 _assetsIn)
    external
    whenNotPaused
-   checkSlippage
    onlyCoreRole(CoreRoles.FARM_SWAP_CALLER)
    returns (bytes32)
{
    // ...
}
function vaultRequestRedeem(uint256 _unitsIn)
    external
    whenNotPaused
-   checkSlippage
    onlyCoreRole(CoreRoles.FARM_SWAP_CALLER)
    returns (bytes32)
{
    // ...
}
function vaultRequestRefund(bytes32 _hash)
    external
    whenNotPaused
-   checkSlippage
    onlyCoreRole(CoreRoles.FARM_SWAP_CALLER)
{
    // ...
}
```

3S-Protocol-N04

Redundant **duration** assignment in **RWAEscrowFarm** constructor

Id	3S-Protocol-N04
Classification	None
Category	Suggestion
Status	Addressed in #93bb574 .

Description

The **RWAEscrowFarm** contract inherits from **MaturityFarm** and is responsible for managing real-world asset (RWA) escrow deposits within the protocol. The constructor initializes the farm by calling the parent **MaturityFarm** constructor and setting up an **RWAEscrow** instance.

The **duration** state variable is declared as **immutable** in the **MaturityFarm** base contract and is already initialized within the **MaturityFarm** constructor. However, the **RWAEscrowFarm** constructor redundantly assigns the same value to **duration** again after the parent constructor has already completed this initialization.

This redundant assignment has no functional impact since both assignments use the same **_duration** parameter value. The duplicate write wastes gas during deployment and reduces code clarity by suggesting that the assignment is necessary when it is not.

Recommendation

Remove the redundant **duration** assignment from the **RWAEscrowFarm** constructor since the value is already set by the **MaturityFarm** parent constructor.

```

    constructor(address _core, address _assetToken, uint256 _duration, address
    _receiver, address _keeper)
    {
        MaturityFarm(_core, _assetToken, _duration, true)
    }
    - duration = _duration;
      escrow = new RWAEscrow(_core, _assetToken, _receiver, _keeper);
      IERC20(assetToken).forceApprove(address(escrow), type(uint256).max);
    }

```

3S-Protocol-N05

Incorrect struct type used for redemption request decoding in **MidasFarm**

Id	3S-Protocol-N05
Classification	None
Category	Suggestion
Status	Addressed in #f5cfcf1 .

Description

The **MidasFarm** contract integrates with Midas vaults to enable deposit and redemption operations. The **assets()** function calculates the total assets held by the farm, including any pending deposit or redemption requests, to provide an accurate accounting of the farm's value. Additionally, **vaultDepositRequest()** and **vaultRedeemRequest()** check for existing pending requests before creating new ones.

When querying pending redemption requests via **IMidasRedeemVault(redeemVault).redeemRequests()**, the contract decodes the returned data using **IMidasManageableVault.Request**. However, this struct definition is intended only for deposit requests. The Midas protocol's redemption vault uses a different struct with distinct field names:

Deposit Request struct (currently used for both):

```
struct Request {
    address sender;
    address tokenIn;
    RequestStatus status;
    uint256 depositedUsdAmount;
    uint256 usdAmountWithoutFees;
    uint256 tokenOutRate;
}
```

Actual Redeem Request struct (from Midas contracts):

```
struct Request {
    address sender;
    address tokenOut;
    RequestStatus status;
```

```

uint256 amountMToken;
uint256 mTokenRate;
uint256 tokenOutRate;
}

```

The code does not revert because both structs share identical field counts and types, making them ABI-compatible. However, the semantic meaning of the fields differs: **depositedUsdAmount** in the deposit struct corresponds to **amountMToken** in the redeem struct. This creates confusion and may lead to incorrect calculations if the field semantics diverge or if future modifications assume correct field names.

Recommendation

Separate the struct definitions for deposit and redemption requests in **IMidasManageableVault.sol**, update the vault interfaces to use the correct types, and update all usages in **MidasFarm.sol**.

```

// IMidasManageableVault.sol
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;
interface IMidasManageableVault {
    enum RequestStatus {
        Pending,
        Processed,
        Canceled
    }
-   struct Request {
+   struct DepositRequest {
        address sender;
        address tokenIn;
        RequestStatus status;
        uint256 depositedUsdAmount;
        uint256 usdAmountWithoutFees;
        uint256 tokenOutRate;
    }
+
+   struct RedeemRequest {
+       address sender;
+       address tokenOut;
+       RequestStatus status;
+       uint256 amountMToken;
+       uint256 mTokenRate;

```

```

+    uint256 tokenOutRate;
+ }
}

```

// IMidasDepositVault.sol

```

interface IMidasDepositVault is IMidasManageableVault {
-   function mintRequests(uint256 id) external view returns (Request memory request);
+   function mintRequests(uint256 id) external view returns (DepositRequest memory request);
    function depositInstant(address tokenIn, uint256 amountToken, uint256 minReceiveAmount, bytes32 referrerId) external;
    function depositRequest(address tokenIn, uint256 amountToken, bytes32 referrerId) external returns (uint256 id);
}

```

// IMidasRedeemVault.sol

```

interface IMidasRedeemVault is IMidasManageableVault {
-   function redeemRequests(uint256 id) external view returns (Request memory request);
+   function redeemRequests(uint256 id) external view returns (RedeemRequest memory request);
    function redeemInstant(address tokenOut, uint256 amountMTokenIn, uint256 minReceiveAmount) external;
    function redeemRequest(address tokenOut, uint256 amountMTokenIn) external returns (uint256 id);
}

```

// MidasFarm.sol - assets() function

```

    // add pending deposit request amount if still pending
    uint256 _pendingDepositRequestId = pendingDepositRequestId;
    if (_pendingDepositRequestId != 0) {
-        IMidasManageableVault.Request memory depositRequest =
+        IMidasManageableVault.DepositRequest memory depositRequest =

IMidasDepositVault(depositVault).mintRequests(_pendingDepositRequestId);
        if (depositRequest.status ==
IMidasManageableVault.RequestStatus.Pending) {
            // deposit amount is in assetToken but normalized to 18 decimals

```

```

        totalAssets += depositRequest.depositedUsdAmount /
decimalsScalingFactor;
    }
}
// add pending redeem request amount if still pending
uint256 _pendingRedeemRequestId = pendingRedeemRequestId;
if (_pendingRedeemRequestId != 0) {
-     IMidasManageableVault.Request memory redeemRequest =
+     IMidasManageableVault.RedeemRequest memory redeemRequest =

IMidasRedeemVault(redeemVault).redeemRequests(_pendingRedeemRequestId);
    if (redeemRequest.status ==
IMidasManageableVault.RequestStatus.Pending) {
        // redeem amount is in mToken
-     totalAssets += convert(mToken, assetToken,
redeemRequest.depositedUsdAmount);
+     totalAssets += convert(mToken, assetToken,
redeemRequest.amountMToken);
    }
}

```

```

// MidasFarm.sol - vaultDepositRequest() function
// prevent new request if there's already a pending one
uint256 _pendingDepositRequestId = pendingDepositRequestId;
if (_pendingDepositRequestId != 0) {
-     IMidasManageableVault.Request memory request =
+     IMidasManageableVault.DepositRequest memory request =

IMidasDepositVault(depositVault).mintRequests(_pendingDepositRequestId);
    require(
        request.status != IMidasManageableVault.RequestStatus.Pending,
        RequestInProgress(_pendingDepositRequestId)
    );
}

```

```

// MidasFarm.sol - vaultRedeemRequest() function
// prevent new request if there's already a pending one
uint256 _pendingRedeemRequestId = pendingRedeemRequestId;
if (_pendingRedeemRequestId != 0) {
-     IMidasManageableVault.Request memory request =
+     IMidasManageableVault.RedeemRequest memory request =

```

```
IMidasRedeemVault(redeemVault).redeemRequests(_pendingRedeemRequestId);  
    require(  
        request.status != IMidasManageableVault.RequestStatus.Pending,  
        RequestInProgress(_pendingRedeemRequestId)  
    );  
}
```